Integration Guide for AI Agent Builders

Give any AI agent a wallet and let it pay for services per call — works with any framework or platform.

Works with: AutoGPT · LangChain · CrewAI · OpenAI Agents · Flowise · MCP clients

Networks: Polygon & Solana mainnet

Model: Non-custodial · no KYC

> **AiFinPay is Stripe for autonomous AI agents.** One line of code — `agent.pay(url)` — lets any agent hold a wallet, pay for any service per call, and settle a real on-chain payment, then receive the gated response. This guide shows the three ways to integrate, leading with the universal one: **MCP**.

01 Why this fits any autonomous agent

Autonomous agents run unattended and call paid services — inference, search, vector DBs, APIs. Today each needs a human's API key and credit card. With AiFinPay, the **agent itself** carries a wallet and pays per call — the missing money layer for agents that do real work.

Each agent gets its own address, a USDC balance, a spend limit, and a transaction history of what it spent on which service.

02 Option 1 — MCP (recommended · zero-code)

Any MCP-compatible client or agent platform — AutoGPT, Claude Desktop, Cursor, Windsurf, Continue, Cline — gains five payment tools with one config block. No SDK calls to wire by hand.

```
// claude_desktop_config.json  /  ~/.cursor/mcp.json
{
  "mcpServers": {
    "aifinpay": {
      "command": "npx",
      "args": ["@aifinpay/mcp"]
    }
  }
}
```

The model now has these tools:

Tool	What it does
`agent_address`	Returns the agent's wallet address (to fund / display)
`agent_quote`	Price-checks an x402-gated URL before paying
`payable_fetch`	Fetches a URL; auto-settles the 402 challenge on-chain

`pay_with_split`	Direct B2B payment with the on-chain split
`quote_split`	Previews the split (merchant / treasury / fee)

That's the whole integration — the agent can now autonomously pay any x402-gated API.

03 Option 2 — Python / Node SDK

```
# Python
pip install aifinpay-agent --pre

# Node / TypeScript
npm install @aifinpay/agent@alpha
```

The “agent pays in a few lines” example — the core of the product:

```python
from aifinpay import Agent

agent = Agent.new()                          # fresh wallet / keypair
print("Fund this address:", agent.address)

# Autonomously settles the 402 challenge on-chain, returns the response
resp = agent.pay(
    "https://bridge.aifinpay.company/io-net/chat/completions",
    body={"model": "meta-llama/Llama-3.3-70B-Instruct",
          "messages": [{"role": "user", "content": "Hello"}]},
)
print(resp.json()["choices"][0]["message"]["content"])
```

Node is identical: `Agent.new()` → `agent.pay(url, { body })`. Persist `agent.secret_b58` for a durable identity (`Agent.from_secret(...)`), and read balance with `agent.balance(asset="USDC", chain="polygon")`.

04 Option 3 — Framework adapters (drop-in)

The SDK ships ready-made examples for the major agent frameworks — clone and run from `github.com/AiFinPay/sdk → examples/`:

Framework	Example
AutoGPT-style headless loop	`examples/autogpt/` (self-funding agent on a budget)
LangChain	`examples/langchain/`
CrewAI	`examples/crewai/`
OpenAI Agents	`examples/openai-agent/`

Flowise	`examples/flowise/`
Claude (MCP)	`examples/claude-mcp/`

05 Under the hood — settlement & events

```
agent → SDK / MCP → x402 challenge (HTTP 402) → agent signs + pays
    → on-chain atomic split → gated response returned to agent
```

- Settlement **is** the transaction — non-custodial, no intermediate hold.
- Each successful call emits one on-chain `Payment` event on the verified splitter contract.
- The protocol fee (1%) is collected automatically by the splitter — no revenue-share bookkeeping.

Webhooks (real-time events)

Subscribe a URL to events and receive HMAC-signed deliveries with automatic retries: events `payment.intent`, `payment.completed`, `seat.reserved`, `b2b.pay_with_split.invoice`. `POST /api/webhooks` to subscribe; each delivery is signed (`x-aifinpay-signature: t=<ts>,v1=<hmac>`).

06 Becoming a paid service (sell to agents)

Want agents to pay *you*? Stand up a paid bridge in ~30 minutes and earn per call — full steps in the SDK's `PARTNER_ONBOARDING.md`. You keep your API key and upstream relationship; AiFinPay handles the payment rail and agent identity.

07 Reference

Resource	Where
SDK + all framework examples	`github.com/AiFinPay/sdk`
Quickstart (60-second first paid call)	`QUICKSTART.md`
MCP client matrix	`MCP_CONFIG.md`
Partner onboarding (sell a service)	`PARTNER_ONBOARDING.md`
Live site	aifinpay.company

Suggested first step

- Add the MCP block (Option 1) — instant payment tools for your agent.
- Run the example for your framework (`examples/`) to see a paid call live.
- Ping us to design an embedded wallet UI (balance + spend limit + per-service history).